ROS2 기초편

#30 Depth 카메라

Calibration

CONTENTS

01 Calibration

02 데이터 수집

03 실습 테스트

캘리브레이션이란 ?

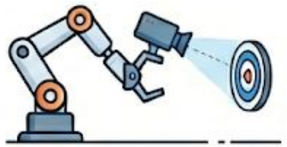
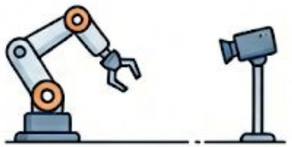
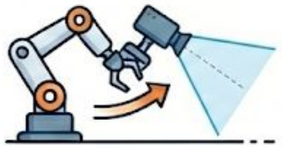
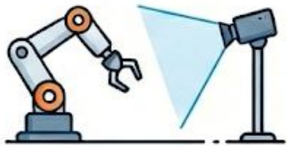
- 카메라의 내·외부 파라미터를 추정하여 정확한 위치 정보를 얻는 과정

캘리브레이션 파라미터

- 내부(Intrinsic): 초점 거리, 주점, 렌즈 왜곡 계수 등 카메라 자체의 왜곡을 보정하는 정보, 수업에서 사용하는 Depth 카메라는 제조사에서 제공하는 보정 값을 그대로 활용
- 외부(Extrinsic): 로봇 좌표계 기준으로 카메라가 위치한 3D 공간 좌표를 매핑하는 정보

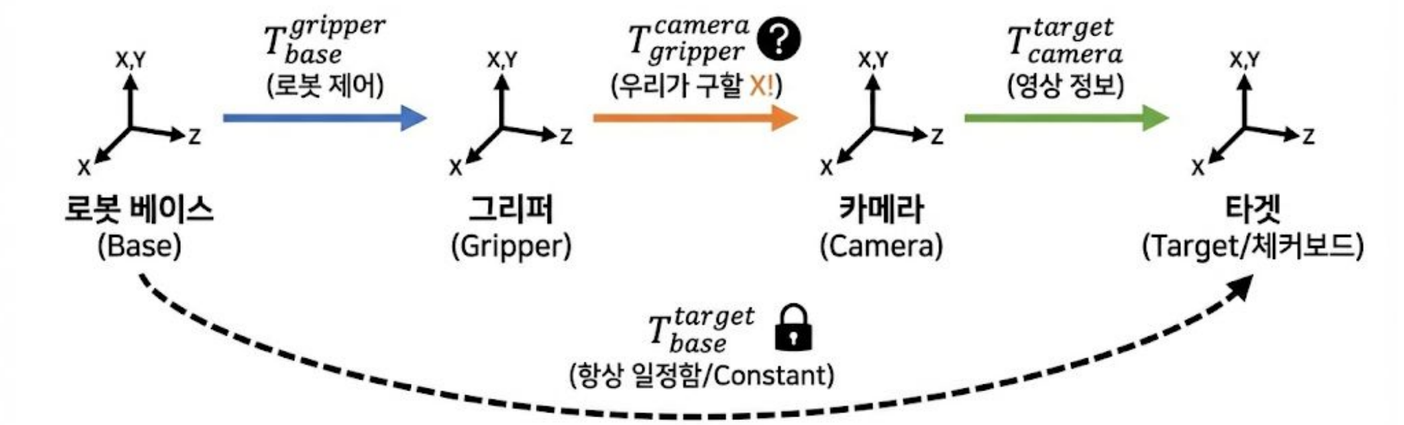
외부 파라미터

- 카메라를 어디에 설치하느냐에 따라 크게 두 가지 방식으로 나뉨

구분	Eye-in-Hand (우리 수업 방식! ✨)	Eye-to-Hand
설치 위치	 카메라를 로봇 팔(그리퍼)에 직접 부착	 카메라를 로봇과 독립적인 고정된 장소에 설치
특징	 로봇이 움직일 때마다 카메라 시점도 함께 이동	 로봇이 움직여도 카메라의 시점은 고정됨
활용	 좁은 공간 작업이나 정밀한 그리핑에 유리	 전체 작업 환경을 조망하거나 넓은 범위를 감시할 때 유리

Hand-Eye Calibration

- 현재 풀어야 할 숙제는 바로 이 체인 룰(Chain-rule) 방정식을 해결하는 것



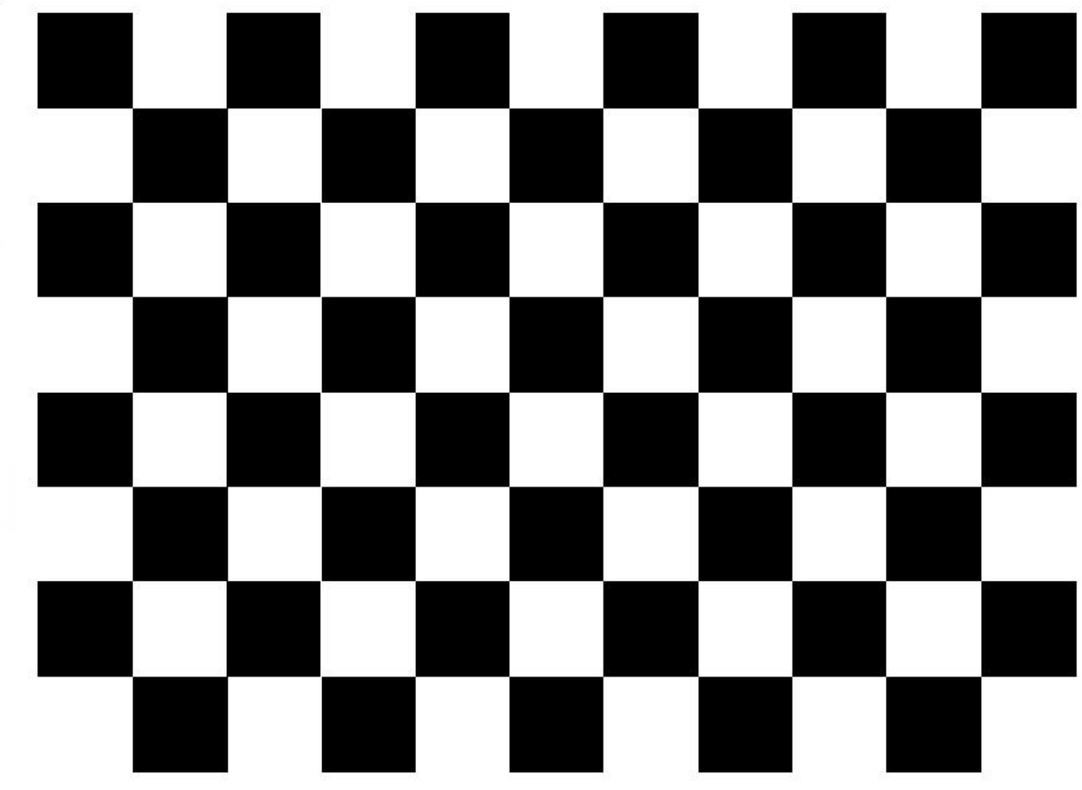
$$T_{base}^{gripper} \cdot T_{gripper}^{camera} \cdot T_{camera}^{target} = T_{base}^{target}$$

Hand-Eye Calibration

- 로봇 기준으로 움직이지 않는 타겟(체커보드)을 여러 자세에서 찍어 얻은 값으로 Hand–Eye 캘리브레이션을 수행하고, 계산된 변환 행렬을 이용해 카메라 좌표를 로봇 베이스 좌표로 변환

체커보드 준비

- 자료 링크 : https://drive.google.com/file/d/1UH4-VaSCKPl3C4xBfQ-8NNF0JOnlAPu6/view?usp=sharing
- 체커보드를 두산 로봇 작업공간에 고정

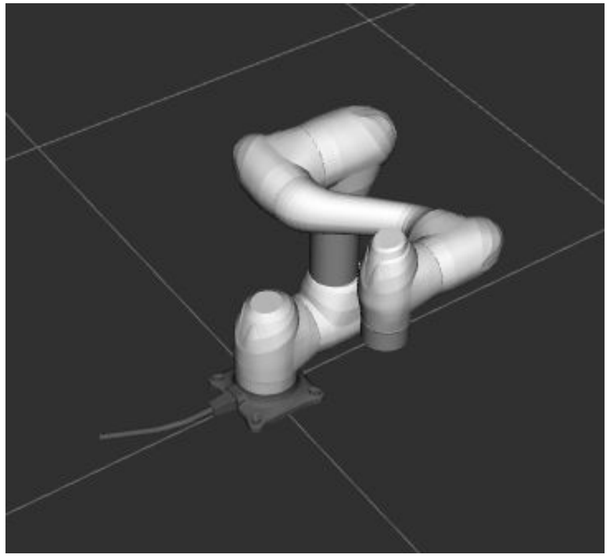


두산 로봇 bringup

- 다음 명령어를 이용해 로봇 bringup 연결하기

```
ros2 launch dsr_bringup2 dsr_bringup2_rviz.launch.py mode:=real model:=m0609 host:=192.168.1.100
```

```
deeptree@robotdive:~$ ros2_ws
ros2_ws is activated!
deeptree@robotdive:~$ ros2 launch dsr_bringup2 dsr_bringup2_rviz.launch.py mode:=real host:=192.168.1.100 port:=12345 model:=m0609
```



두산 로봇 bringup

- 다른 터미널을 열어 로봇 모드를 직접 교시로 전환
- 로봇 LED가 파란색인지 확인

```
ros2 service call /dsr01/system/set_robot_mode dsr_msgs2/srv/SetRobotMode "robot_mode: 0"
```

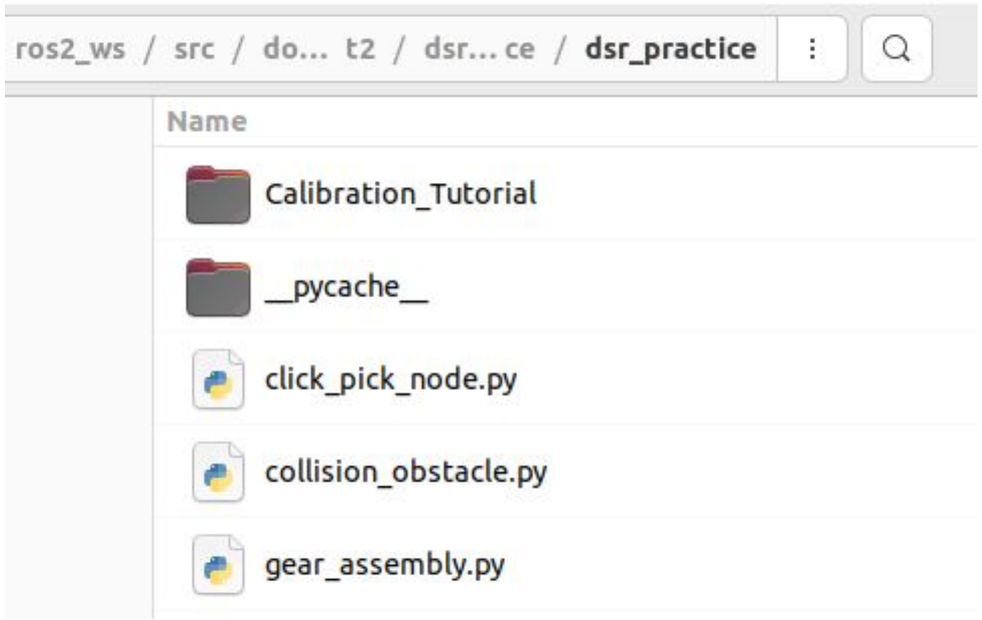
```
deeptree@robotdive:~$ ros2_ws
ros2_ws is activated!
deeptree@robotdive:~$ ros2 service call /dsr01/system/set_robot_mode dsr_msgs2/srv/SetRobotMode "robot_mode: 0"
requester: making request: dsr_msgs2.srv.SetRobotMode_Request(robot_mode=0)

response:
dsr_msgs2.srv.SetRobotMode_Response(success=True)
```

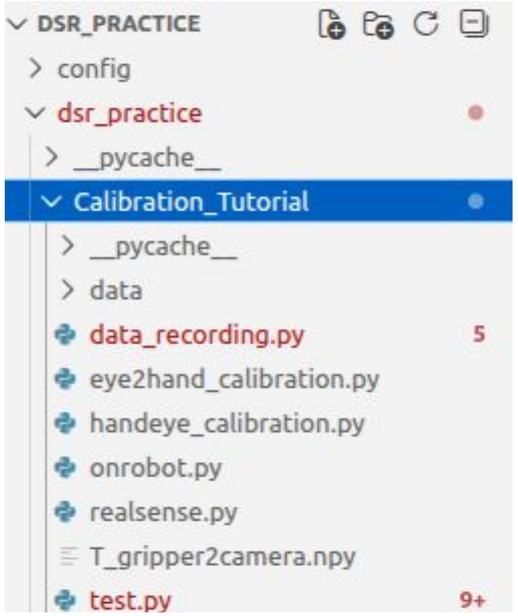
- 원격 제어 모드 전환

```
ros2 service call /dsr01/system/set_robot_mode dsr_msgs2/srv/SetRobotMode "robot_mode: 1"
```

자료 다운

- 캘리브레이션 자료 수집을 위한 코드 다운로드
- Zip파일 다운로드 후 /ros2_ws/src/doosan-robot2/dsr_practice/dsr_practice 에 압축해제





v4l-utils 설치

```
sudo apt install v4l-utils
```

DEVICE_NUMBER 확인

- Depth 카메라 번호를 확인하고 video 뒤 숫자들을 다시 확인
- Pixel Format와 Colorspace가 일치하면 해당 video 번호의 카메라 사용

```
deeptree@robotdive:~$ v4l2-ctl --list-devices
LG Camera: LG Camera (usb-0000:00:14.0-2):
        /dev/video0
        /dev/video1
        /dev/media0

Intel(R) RealSense(TM) Depth Ca (usb-0000:00:14.0-3):
        /dev/video2
        /dev/video3
        /dev/video4
        /dev/video5
        /dev/video6
        /dev/video7
        /dev/media1
        /dev/media2
```

```
deeptree@robotdive:~$ v4l2-ctl -d /dev/video6 --get-fmt-video
Format Video Capture:
        Width/Height      : 320/180
        Pixel Format      : 'YUYV' (YUYV 4:2:2)
        Field             : None
        Bytes per Line    : 640
        Size Image        : 115200
        Colorspace        : sRGB
        Transfer Function : Default (maps to sRGB)
        YCbCr/HSV Encoding: Default (maps to ITU-R 601)
        Quantization      : Default (maps to Limited Range)
        Flags             :
```

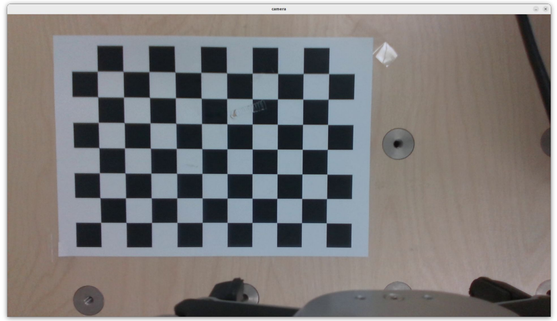
data_recording.py 실행

- Calibration_Tutorial 폴더에 있는 data_recording.py에서 DEVICE_NUMBER 수정(video* 번호 중 컬러 선택)
- 새로운 터미널에서 CalibrationTutorial 폴더에 있는 data_recording.py 파일 실행
- 실행 화면에서 q를 누르면 현재 찍고 있는 사진과 좌표 값이 저장됨
- 직접 교시로 위치를 바꾸어 가며 데이터 수집(20장 이상 수집)

```
export PYTHONPATH=$PYTHONPATH:~/ros2_ws/install/dsr_common2/lib/dsr_common2/imp
cd ~/ros2_ws/src/doosan-robot2/dsr_practice/dsr_practice/Calibration_Tutorial/
```

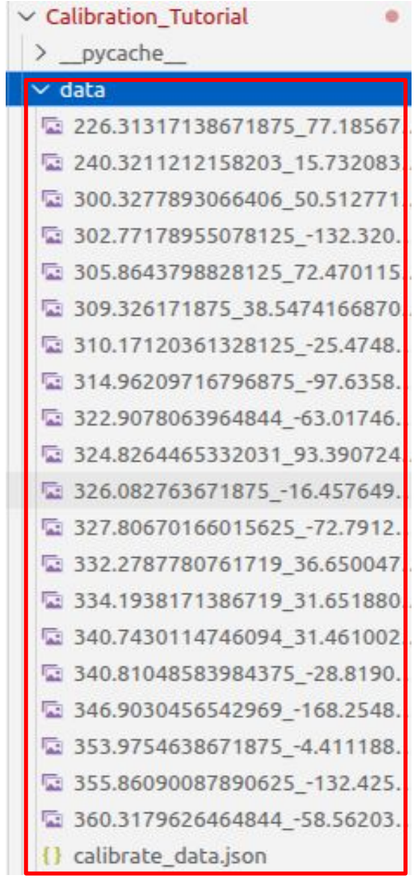
```
# 로봇 설정
ROBOT_ID = "dsr01"
ROBOT_MODEL = "m0609"
VELOCITY, ACC = 60, 60
DEVICE_NUMBER = 6

DR_init.dsr__id = ROBOT_ID
DR_init.__dsr__model = ROBOT_MODEL
```

```
deeptree@robotdive:~$ ros2_ws
ros2_ws is activated!
deeptree@robotdive:~$ export PYTHONPATH=$PYTHONPATH:~/ros2_ws/install/dsr_common2/lib/dsr_common2/imp
deeptree@robotdive:~$ cd ros2_ws/src/doosan-robot2/dsr_practice/dsr_practice/Calibration_Tutorial/
deeptree@robotdive:~/ros2_ws/src/doosan-robot2/dsr_practice/dsr_practice/Calibration_Tutorial$ python3 data_recording.py
```



data_recording.py 실행

- 데이터 수집 완료 시 아래와 같은 디렉토리 구조가 되어야 함




```
1   {
2       "poses": [
3           [
4               300.3277893066406,
5               50.51277160644531,
6               550.855224609375,
7               156.1127166748047,
8               -173.249755859375,
9               74.10289001464844
10          ],
11          [
12              240.3211212158203,
13              15.732083320617676,
14              554.3858032226562,
15              171.8446044921875,
16              -170.18356323242188,
17              95.73522186279297
18          ],
```

```
164     "file_name": [
165         "300.3277893066406_50.51277160644531_550.855224609375.jpg",
166         "240.3211212158203_15.732083320617676_554.3858032226562.jpg",
167         "226.31317138671875_77.18567657470703_546.2445678710938.jpg",
168         "305.8643798828125_72.4701156616211_543.5835571289062.jpg",
169         "309.326171875_38.54741668701172_542.5922241210938.jpg",
170         "326.082763671875_-16.45764923095703_542.4790649414062.jpg",
171         "327.80670166015625_-72.791259765625_532.1045532226562.jpg",
172         "334.1938171386719_31.651880264282227_527.7757568359375.jpg",
173         "340.7430114746094_31.461002349853516_454.61956787109375.jpg",
```

캘리브레이션 계산

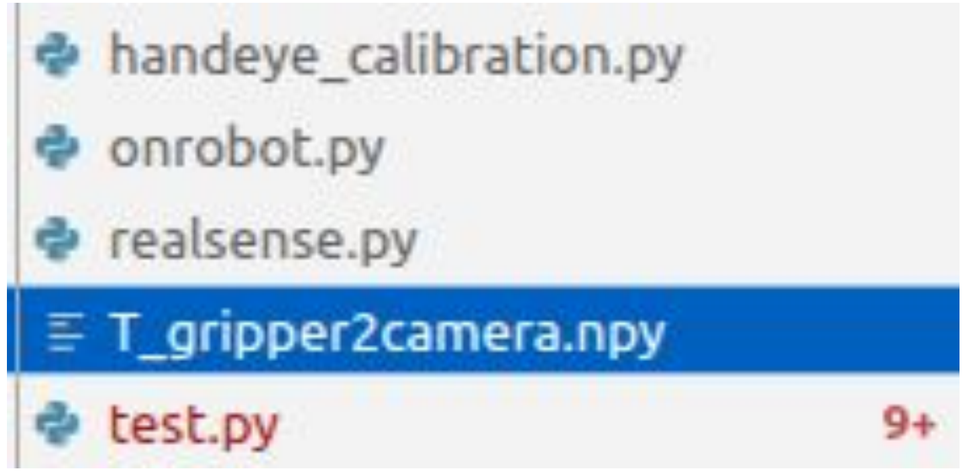
- handeye_calibration.py 에서 checkerboard_size = (10, 7) square_size = 25 로 수정

```python
# Main Function
if __name__ == "__main__":
    # 캘리브레이션 데이터 로드
    data = json.load(open("data/calibrate_data.json"))
    robot_poses = np.array(data["poses"])

    robot_poses[:, :3] = robot_poses[:, :3]
    image_paths = ["data/" + d for d in data["file_name"]]

    checkerboard_size = (10, 7)  # 내부 코너 개수
    square_size = 25
    # 카메라 캘리브레이션 수행(내부 파라미터 왜곡 보정)
    camera_matrix, dist_coeffs, rvecs, tvecs = calibrate_camera_from_chessboard(
        image_paths, checkerboard_size, square_size
    )
```

캘리브레이션 계산

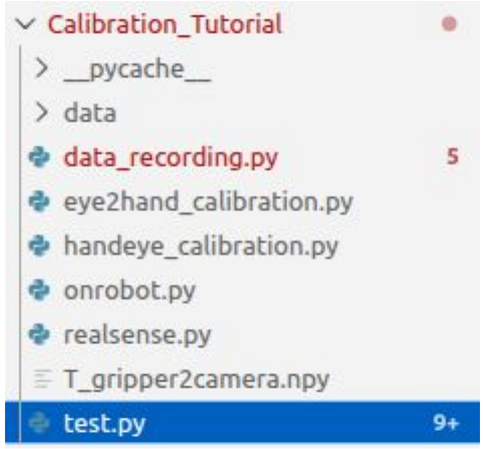
- handeye_calibration.py 파일 실행하기
- 실행 후 T_gripper2camera.npy 파일이 만들어졌는지 체크하기

```
deeptree@robotdive:~/ros2_ws/src/doosan-robot2/dsr_practice/dsr_practice/Calibration_Tutorial$ python3 handeye_calibration.py
```



test.py 작성

- 생성된 T_gripper2camera.npy 파일을 검증하기 위한 test.py 생성
- 카메라 클릭으로 얻은 픽셀을 Depth 기반 3D 좌표로 변환하고, Hand–Eye 행렬을 통해 베이스 좌표로 변환한 뒤 로봇 Pick & Place를 수행



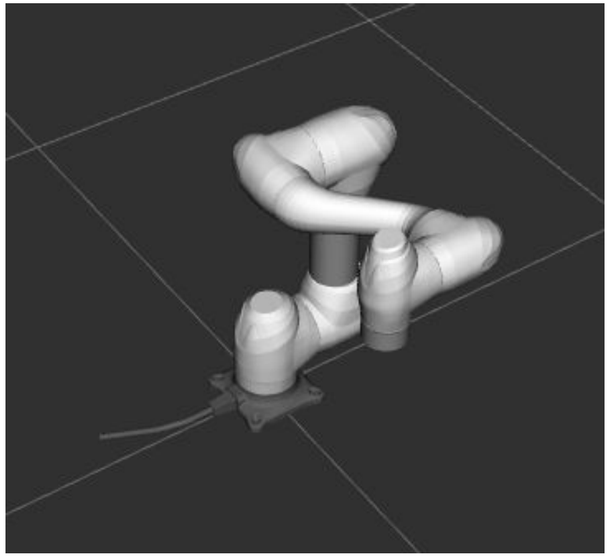

```python
import cv2
import rclpy
import time
import numpy as np
import threading
from scipy.spatial.transform import Rotation

from realsense import ImgNode
from onrobot import RG
import DR_init

# =======================
# 로봇 / 그리퍼 설정
# =======================
ROBOT_ID = "dsr01"
ROBOT_MODEL = "m0609"
VELOCITY, ACC = 60, 60

DR_init.__dsr__id = ROBOT_ID
DR_init.__dsr__model = ROBOT_MODEL

GRIPPER_NAME = "rg2"
TOOLCHARGER_IP = "192.168.1.1"
TOOLCHARGER_PORT = 502  # 정수

# =======================
# Z 관련 파라미터
# =======================
Z_OFFSET     = 280.0   # 클릭한 지점 z에 더해 줄 오프셋 (mm)
SAFE_Z       = 400.0   # 집은 뒤 올라갈 안전 높이 (mm) – 환경 보고 조정
```

두산 로봇 bringup

- 다음 명령어를 이용해 로봇 bringup rviz 연결하기

```
ros2 launch dsr_bringup2 dsr_bringup2_rviz.launch.py mode:=real model:=m0609 host:=192.168.1.100
```

```
deeptree@robotdive:~$ ros2_ws
ros2_ws is activated!
deeptree@robotdive:~$ ros2 launch dsr_bringup2 dsr_bringup2_rviz.launch.py mode:=real host:=192.168.1.100 port:=12345 model:=m0609
```



Realsense 런치 파일 실행

- 새로운 터미널을 열고 다음 명령어를 이용해 Depth 카메라 런치 파일 실행

```
ros2 launch realsense2_camera rs_align_depth_launch.py depth_module.depth_profile:=640x480x30
rgb_camera.color_profile:=640x480x30 initial_reset:=true align_depth.enable:=true
```

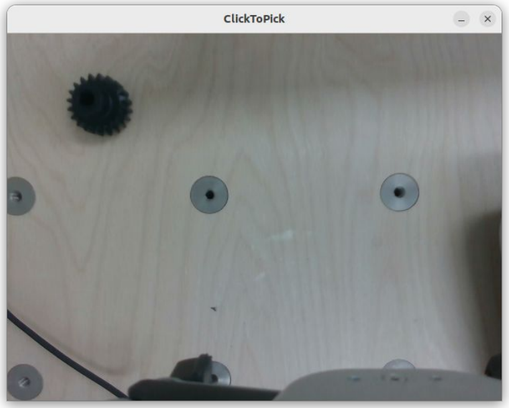
```
ssu@ssu-22663-34:~/ros2_ws$ ros2 launch realsense2_camera rs_align_depth_launch.py depth_module.depth_
profile:=640x480x30 rgb_camera.color_profile:=640x480x30 initial_reset:=true align_depth.enable:=true
```

test.py 실행

- 새로운 터미널을 열고 다음 명령어를 이용해 test.py 실행
- 화면을 클릭해 Pick and Place가 제대로 되는지 확인
- Z축 관련 파라미터들을 수정하면서 정확도 올리기

```
export PYTHONPATH=$PYTHONPATH:~/ros2_ws/install/dsr_common2/lib/dsr_common2/imp
python3 test.py
```

```
deeptree@robotdive:~$ ros2_ws
ros2_ws is activated!
deeptree@robotdive:~$ export PYTHONPATH=$PYTHONPATH:~/ros2_ws/install/dsr_common2/lib/dsr_common2/imp
deeptree@robotdive:~$ cd ros2_ws/src/doosan-robot2/dsr_practice/dsr_practice/Calibration_Tutorial/
deeptree@robotdive:~/ros2_ws/src/doosan-robot2/dsr_practice/dsr_practice/Calibration_Tutorial$ export PYTHONPATH=$PYTHONPATH:
~/ros2_ws/install/dsr_common2/lib/dsr_common2/imp
deeptree@robotdive:~/ros2_ws/src/doosan-robot2/dsr_practice/dsr_practice/Calibration_Tutorial$ ros2_ws
ros2_ws is activated!
deeptree@robotdive:~/ros2_ws/src/doosan-robot2/dsr_practice/dsr_practice/Calibration_Tutorial$ python3 test.py
```



click_pick_node 생성

- 카메라 화면에서 마우스로 클릭하면, 로봇팔이 해당 물체를 집어서 홈 위치에 내려놓는 Pick & Place 노드

동작 흐름

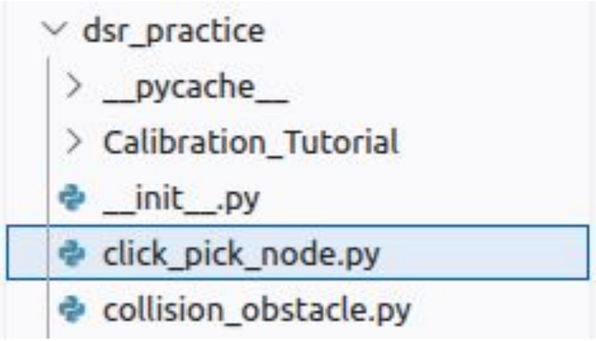
1. **카메라 영상 수신** → OpenCV 윈도우에 표시
2. **마우스 클릭** → 클릭한 픽셀의 Depth 값으로 3D 좌표 계산
3. **좌표 변환** → Hand-Eye 캘리브레이션 행렬로 카메라 좌표 → 로봇 베이스 좌표 변환
4. **MoveIt으로 Pick & Place 실행** → 물체 집기 → 홈 위치에 내려놓기

주요 구성 요소

- **MoveItPy** : 로봇팔 경로 계획 및 실행
- **RealSense 카메라** : 컬러 + Depth 이미지 수신
- **RG2 그리퍼** : 물체 파지/해제
- **Hand-Eye 캘리브레이션** : 카메라-로봇 좌표계 연결

click_pick_node 생성

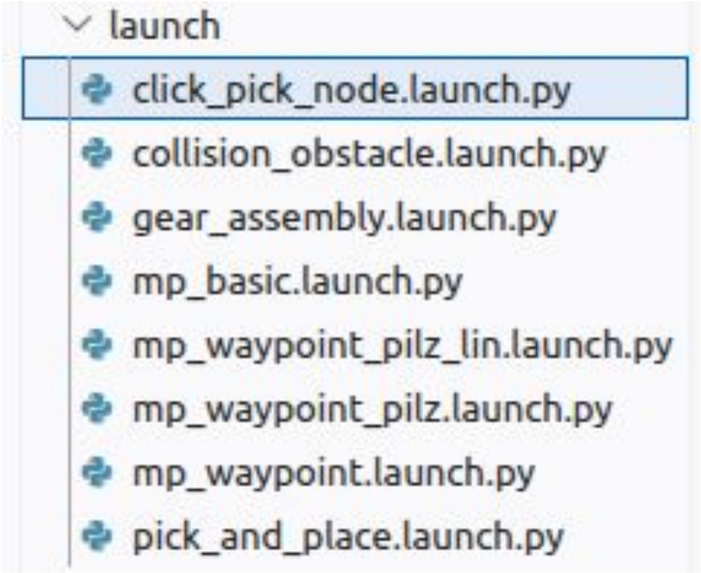
- /home/ssu/ros2_ws/src/doosan-robot2/dsr_practice/dsr_practice에 click_pick_node.py 생성




```python
import math
import time
from pathlib import Path

import cv2
import numpy as np
import rclpy
from rclpy.node import Node
from rclpy.logging import get_logger

from scipy.spatial.transform import Rotation
from ament_index_python.packages import get_package_share_directory

from geometry_msgs.msg import PoseStamped
from sensor_msgs.msg import Image, CameraInfo
from cv_bridge import CvBridge

from moveit.core.robot_state import RobotState
from moveit.planning import MoveItPy, PlanRequestParameters

from .onrobot import RG


# ────────────────────────────────────────────────
#  MoveIt / 로봇 설정
# ────────────────────────────────────────────────
GROUP_NAME = "manipulator"
BASE_FRAME = "base_link"
EE_LINK    = "link_6"
```

click_pick_node.launch.py 생성

- launch 파일 생성




```python
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.substitutions import PathJoinSubstitution
from launch_ros.substitutions import FindPackageShare
from moveit_configs_utils import MoveItConfigsBuilder

def generate_launch_description():
    # Doosan M0609 MoveIt 기본 설정 (URDF, SRDF, kinematics, controllers 등)
    moveit_config = (
        MoveItConfigsBuilder(
            robot_name="m0609",
            package_name="dsr_moveit_config_m0609",
        )
        .robot_description()                # m0609.urdf.xacro
        .robot_description_semantic(file_path="config/dsr.srdf")  # dsr.srdf
        .robot_description_kinematics()     # kinematics.yaml
        .joint_limits()                     # joint_limits.yaml
        .trajectory_execution()             # moveit_controllers.yaml
        .planning_scene_monitor()           # sensors_3d.yaml 등과 연동
        .sensors_3d()
        .to_moveit_configs()
    )

    # 🔹 MoveItPy 전용 YAML 추가
    moveit_py_params = PathJoinSubstitution(
        [FindPackageShare("dsr_practice"), "config", "moveit_py.yaml"]
    )

    return LaunchDescription(
        [
            Node(
                package="dsr_practice",
                executable="click_pick_node",
                output="screen",
                # MoveIt config + MoveItPy용 설정을 같이 넘김
                parameters=[
                    moveit_config.to_dict(),
                    moveit_py_params,
                ],
            )
        ]
    )
```

setup.py 수정

```python
from setuptools import find_packages, setup
from glob import glob

package_name = 'dsr_practice'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name + '/launch', glob('launch/*.launch.py')),
        (
            'share/' + package_name + '/config',
            glob('config/*.yaml') + glob('dsr_practice/Calibration_Tutorial/*.npy')
        ),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='deeptree',
    maintainer_email='deeptree@todo.todo',
    description='TODO: Package description',
    license='TODO: License declaration',
    extras_require={
        'test': [
            'pytest',
        ],
    },
    entry_points={
        'console_scripts': [
            'mp_basic = dsr_practice.mp_basic:main',
            'mp_waypoint = dsr_practice.mp_waypoint:main',
            'mp_waypoint_pilz = dsr_practice.mp_waypoint_pilz:main',
            'mp_waypoint_pilz_lin = dsr_practice.mp_waypoint_pilz_lin:main',
            'collision_obstacle = dsr_practice.collision_obstacle:main',
            'gripper = dsr_practice.gripper:main',
            'gear_assembly = dsr_practice.gear_assembly:main',
            'click_pick_node = dsr_practice.click_pick_node:main',
        ],
    },
)
```

click_pick_node 실행

- [터미널 1] moveit bringup 실행
- [터미널 2] 카메라 런치
- [터미널 3]빌드 후 click_pick_node 런치

```
ros2 launch dsr_bringup2 dsr_bringup2_moveit.launch.py mode:=real model:=m0609 host:=192.168.1.100
ros2 launch realsense2_camera rs_align_depth_launch.py depth_module.depth_profile:=640x480x30 rgb_camera.color_profile:=640x480x30 initial_reset:=true align_depth.enable:=true
ros2 launch dsr_practice click_pick_node.launch.py
```

```
ssu@ssu-22663-34:~/ros2_ws$ ros2 launch dsr_bringup2 dsr_bringup2_moveit.launch.py mode:=real model:=
m0609 host:=192.168.1.100
```

```
ssu@ssu-22663-34: ~ 101x5
ssu@ssu-22663-34:~/ros2_ws$ ros2 launch realsense2_camera rs_align_depth_launch.py depth_module.depth_
profile:=640x480x30 rgb_camera.color_profile:=640x480x30 initial_reset:=true align_depth.enable:=true
```

```
ssu@ssu-22663-34: ~/ros2_ws 101x12
ssu@ssu-22663-34:~/ros2_ws$ colcon build --packages-select dsr_practice
Starting >>> dsr_practice
Finished <<< dsr_practice [0.43s]

Summary: 1 package finished [0.58s]
ssu@ssu-22663-34:~/ros2_ws$ ros2_ws
ros2_ws is activated!
ssu@ssu-22663-34:~/ros2_ws$ ws_moveit
ws_moveit is activated!
ssu@ssu-22663-34:~/ros2_ws$ ros2 launch dsr_practice click_pick_node.launch.py
```

click_pick_node 실행

- 화면에 있는 물건을 클릭 후 움직임 관찰
- 필요에 따라 Z_OFFSET 조절